20. Софтуерна архитектура. Проектиране и документиране на софтуерни архитектури.
Изложението по въпроса трябва да включва следните по-съществени елементи:
1. Дефиниция на софтуерна архитектура. Структури и изгледи (structures and views) на архитектурата.
2. Изисквания към качеството (нефункционални изисквания) на системата.
3. Проектиране на софтуерната архитектура. Процес за проектиране. Избор на подходящи структури. Последователност на създаване на архитектурата. Тактики (архитектурни решения) за постигане на желаните качествени показатели.
4. Архитектурни стилове.
5. Документиране на софтуерната архитектура. Предназначение на документацията. Основен принцип на документиране. Съдържание на документацията. Структура на документацията

1. **Дефиниция на софтуерна архитектура. Структури и изгледи (structures and views) на архитектурата**.

Архитектурата на дадена софтуерна система е съвкупност от структури, показващи различните софтуерни елементи на системата, външно видимите им свойства и връзките между тях. Софтуерната архитектура е **абстракция**, която разглежда елементите като **черни** кутии (т.е. не се интересува от алгоритми и прочие), като се фокусира над **взаимодействията** между тях

Структура – съвкупност от софтуерни елементи, техните външно видими свойства и връзките между тях.
Изглед – конкретно документирано представяне на дадена структура. Полезни са за комуникация със заинтересованите лица.

Архитектурните структури се делят главно на три групи:
1. **Модулни структури** - описва основните функционални компоненти и техните взаимовръзки. Например, в една система за онлайн магазин, логическите компоненти може да включват модул за продукти, модул за поръчки, модул за потребители и модул за плащания.
2. **Структури на процесите** - описва динамичното поведение на системата, включително как различните компоненти комуникират и си взаимодействат по време на изпълнение. Например, как заявката на потребител за добавяне на продукт в количката преминава през различни слоеве на системата.
3. **Структури на разположението** - показва как логическите компоненти са разпределени върху хардуерните ресурси. В същия пример за онлайн магазин, физическата структура може да включва уеб сървър, база данни сървър, кеш сървър и бекенд сървър.

Архитектурните изгледи се делят главно на 4 групи:
1. **Логически изглед** – показва основните абстракции в системата, като обекти, класове и компоненти. Пример: **диаграма на класовете**, която показва връзките между различните логически модули
2. **Изглед на процесите** – показва системата като съвкупност от взаимодействащи си процеси по време на изпълнение. Пример: **диаграма на**

последователността, която показва как заявката за поръчка на продукт в онлайн преминава през различните компоненти на системата

3. **Изглед на кода** – Показва как отделните елементи на системата се разполагат във файлове код. Пример: **диаграма на пакети**, която показва как различните части на кода са организирани.
4. **Физически изглед** – показва как софтуерните компоненти са разпределени между хардуерните възли в системата. Пример: **диаграма на разполагане**, която показва кои логически компоненти са разположени на кои сървъри или устройства.

2. **Изисквания към качеството (нефункционални изисквания) на системата**.

Освен **функционалните** изисквания, които определят какво трябва да прави софтуерната система, съществуват и **нефункционални** изисквания, които определят как трябва да работи софтуерната система. На практика качествата поставят **ограничения** върху начина, по който системата ще работи. Някои от качествата биват: надеждност, производителност, скалируемост, използваемост, тестваемост, сигурност, преносимост, наличност и др.

Качеството на системата е **субективно понятие** и различните заинтересовани лица имат различни възпричтия за него. Затова изискванията за качество на системата трябва да се нормализират от архитекта, което се постига чрез така наречение **сценарии за качество**.

Пример за сценарий за качество: По време на работа на системата, потребителят прави годишна справка за продажбите в онлайн магазин и системата обработва заявката и връща pdf файл за по-малко от 15 секунди.

Всеки сценарий на качество описва някаква случка и се характеризира с 6 компонента:

- **Контекст** - по време на работа на системата
- **Източник** - потребителят
- **Въздействие** - прави годишна справка за продажбите в онлайн магазин
- **Обект** - системата
- **Резултат** - обработва заявката и връща pdf файл
- **Количествени параметри** - за по-малко от 15 секунди

3. **Проектиране на софтуерната архитектура. Процес за проектиране. Избор на подходящи структури. Последователност на създаване на архитектурата. Тактики (архитектурни решения) за постигане на желаните качествени показатели**.

Процесът на проектиране представлява циклично изпълнение на последователност от стъпки, които водят до изграждането на дадена софтуерна архитектура. Първоначално различните stakeholders имат различни и неструктурирани нужди и искания от системата. Първата стъпка е да анализираме тези искания и нужди и да определим функционалните и нефункционалните изисквания към системата.
Избират се най-приоритетните изисквания, които се превръщат в случаи на употреба (use-cases) при функционалните изисквания и сценарии за качество при нефункционалните изисквания. Това са така наречените архитектурни driver-и. След това се пристъпва към проектиране на системата, където могат да възникнат

проблеми, които от своя страна могат да доведат до промяна на архитектурните драйвери. Ето защо процесът на проектиране е цикличен.

- **Избор на подходяща структура** - зависи от специфичните изисквания и контекста на проекта. Важно е да се намери баланс между различните изисквания и да се изберат структури, които най-добре обслужват системата. Много често се използва декомпозиция на модулите.
- **Последователност на създаване на архитектурата** - след като сме направили анализ на изискванията и имаме: функционални изисквания, като сценарии за употреба; функционални ограничения и сценарии за качество - идентифицираме отделните модули на системата и за всеки един от модулите извършваме следните стъпки:
    - Избират се архитектурни driver-и, които се отнасят за модула
    - Избират се архитектурни тактики (като кеширане, репликация) и шаблони (MVC, микросървиси) за осъществяване на driver-ите
    - Създават се подмоделите на модулите
    - Приписва се функционалност на подмодулите
    - Създават се различни диаграми и модели на подмодулите, ако има нужда (диаграма на класовете, диаграма на последователност, диаграма на разполагане и т.н)
    - Проверява се дали декомпозицията отговаря на изискванията
    - Задълбаваме рекурсивно ако има нужда да дадем повече детайли за някой от подмодулите

Определяме връзките и взаимодействията между модулите и след това е много важно да оценим, валидираме и документираме добре архитектурата.

- Тактики за постигане на желаните качествени характеристики
    - тактики за **надеждност**
        - ping/echo, Healthcheck (компонент А пуска сигнал на модул Б и очаква отговор в определен интервал от време); Heartbeat; Keepalive; активен излишък; пасивен излишък и др.
    - тактики за **производителност**: кеширане; разпределение на натоварването(Load-balancing); паралелна обработка на заявките; премахване на Overhead от различни операции; по-мощен хардуер
    - тактики за **изменяемост**: Open-Closed Solid principle
    - тактики за **сигурност**: криптиране на данните; хеширане (например на пароли); двуфакторна автентикация;
    - тактики за **скалируемост**: хоризонтално скалиране (scaling out); вертикално скалиране (scaling up); разделяне на базата данни (sharding)

4. **Архитектурни стилове**.

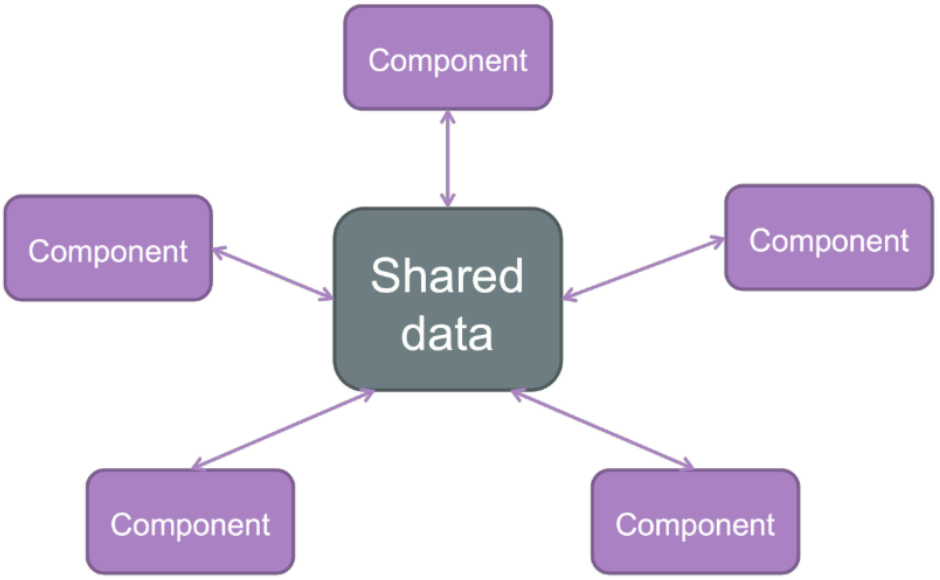
- **Pipe-and-filter**



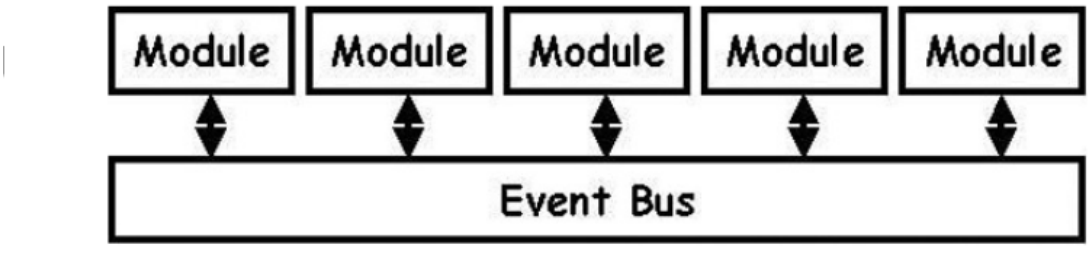
Модулите се делят на филтри, канали, източник на данни и консуматор на данни. Каналите са потоци от данни изпълняващи ролята на конектори между

филтрите. Филтрите са модули, трансформиращи входните си данни, които могат да работят паралелно и независимо едни от други, като всеки филтър започва да предава изходните данни към следващия филтър преди да е свършил с обработката на всичките данни. Този начин за обработка на данни позволява паралелизъм (конвейеризация).
Филтрите нямат никаква информация за съседите си, което прави архитектурата гъвкави лесна за разбиране. От друга страна, производителността страда заради parse-ването на данни от всеки филтър.

- **Слоеста архитектура (Layered)** - компонентите са вертикално разделени слоеве, като всеки слой може да предлага услуга на слоя над себе си и да използва услугите на слоя под себе си.
- **Клиент-сървър** - системата е изградена като сървър, който предлага ресурси и клиент, който консумира ресурсите.
- **Shared-data** - данните са като конектор между компонентите. Съществуват две вариации на този стил: Repository и Blackboard



- **Implicit invocation/message passing** - компонентите си комуникират чрез така наречените събития. Компонентите работят паралелно, а конектора е шина, по която се предават съобщенията.



- Съществуват и други архитектурни стилове като: MVC, Domain-driven design, Microservices, Service-Oriented Architecture и т.н.

5. **Документиране на софтуерната архитектура. Предназначение на документацията. Основен принцип на документиране. Съдържание на документацията. Структура на документацията**.

a. **Предназначение на документацията:**
    i. **комуникация** - улесняване на комуникацията между различните заинтересовани лица
    ii. **поддръжка и развитие** - да бъде безпроблемно изменяна и лесно поддържана системата
    iii. **служи като основа за разработката на системата**
b. основни принципи на документацията - **да знаем кой ще чете документацията**; човек, който пише документацията трябва да се постави на мястото на човекът, който ще чете документацията. да бъде ясна, точна и добре структурирана
c. **Съдържание на документацията**
    i. Общ преглед на архитектурата - описание на основните структури и компоненти
    ii. Изгледи на архитектурата
        1. Логически изглед
        2. Изглед на кода
        3. Изглед на процесите
        4. Физически изглед
    iii. Нефункционални изисквания (критерии за качество) - описание на ключови критерии за качеството
    iv. Описание на елементите на архитектурата и връзките - детайлно описание на различните елементи и връзката между тях
    v. Архитектурни решения - обосновка за взетите архитектурни решения
    vi. Описание на възможно вариации
    vii. Терминологичен речник
d. **Структура на документацията**
    i. въведение: цели, обхват и общ поглед на архитектурата
    ii. архитектурни изгледи: подробни изгледи на архитектурата
    iii. сценарии за качество
    iv. архитектурни решения - обосновка и описание на взетите архитектурни решения
    v. интеграция и тестване - планове за интеграция и тестване на системата
    vi. поддръжка и развитие - насоки за поддръжка и бъдещо развитие на системата

Записки от консултация 12.6.2024 г.
- може да изпратим по email развита тема за коментари
- само дефиницията за софтуерна архитектура не е достатъчно - трябва да поясним какво е структура, какво са софтуерните елементи, какво са връзките между тях, какво са външно видимите им свойства
- изглед - синоним на структура (различен поглед върху структурата)
- структури - модулни/логически; на процесите; физически/на разположението
- модулите са софтуерните елемент в стуктура на модулте
- външно видимите свойста - интерфейс
- черна-кутия
- елементие на структурата представляват черни кутии, които скриват логиката
- структура на процесите (разглеждаме системата в конкретни състояния на системата, activity diagram)
- физическа структура/на внедряване
- софтуерната архитектура зависи най-много от качествените изисквания
- цел: намиране на баланс между различните качествени изисквания
- въвеждаме сценарии за качеството, за да формализира изискванията за тези качествени харакетеристики
- сценарии за качество - 6 компоннета
- най-важните: резултат и количествени параметри
- пример за сценарии за качество
- проекитрането започва, когато имаме някакви изксвания (може и да не са пълни)
- променят се изискванията => види до промяна и на архитектурата (исикванията също може да променят архитектурата)
- attribute-driven design
- прилагаме конкретни тактики и архитектурни стилове, което да удовлетворят изискванията
- например чрез декомпозиция на модулите можем да разделим системата на по-малки части
- архитектурни driver-и
- разделяме цялата системата херархично на модули
- архутектурни стилове - често срещани решения
- за документиране на СА трябва да се изплзва някакви шаблон
- описание на самата документация (докумнетация за докумнетацията, как да се чете, как да се докунтира)
- съвкупност от документиране поотделно на всяка една от софтуерните структури
- всяка една структура има седем елемента (първична, вторична, опсание на обкръжението, варианти за промяна в структурата)
- терминологичен речник
- допълнително съдържание, забележки

- практическа част: декомпозиция на модули
- в описанието да напишем някаква тактика, с която решаваме проблема (architecture driver-a)

- критични ситуации - арбитраж на ресурсите (допълнителен модул, който да прекъсне текущата задача и да започне да изпълнява handler на критичната ситуация)
-